Mood Controlled Automatic Music Generator

https://github.com/DiyaPrakash/Automatic-Music-Generator

Diya Prakash
2023BCS0028
IIITK

R Nithin Athreya
2023BCS0103
IIITK

Praveen Kumar
2023BCS0115
IIITK

***Abstract*—Automatic music generators are hard to steer toward a requested mood. This project examines a prototype that generates symbolic piano music from a selectable mood label and tempo. Piano performances from the MAESTRO dataset are converted into sequences over 234 event tokens that describe note onsets, note offsets, velocity levels, and time shifts on a 0.1 second grid. Because the dataset has no mood annotations, five weak labels (happy, sad, calm, energetic, and suspenseful) are assigned by hand-written rules over tempo, note density, pitch range, and an estimated major or minor mode. A two-layer long short-term memory (LSTM) network with 1,014,714 parameters predicts the next event from the previous 100 events and a learned mood embedding. Training used a random subset of 300 of the 1,276 available performances, divided by file into 255 training and 45 validation performances, and ran for ten epochs. Validation loss fell to 1.91 and next-event accuracy reached 46.1 percent, and a 500-event sample was decoded into a playable Musical Instrument Digital Interface (MIDI) file. However, the labelling rules assigned all 300 sampled performances to the energetic class, so the other four mood embeddings were never trained and the experiment gives no evidence of mood control. The tempo setting was also found to change only file metadata. The paper documents these findings, their likely causes, and the changes needed for a valid mood-controlled generator.**

Keywords—symbolic music generation, long short-term memory, mood conditioning, weak labels, MAESTRO dataset, MIDI event tokens

I. Introduction

Neural networks can now produce symbolic music that is locally coherent, but making a model respond to a request such as “calm” or “energetic” is considerably harder than making it produce plausible notes [1], [2]. Mood is a property perceived by listeners. Large performance corpora rarely annotate it, and the cues that carry it (tempo, density, register, harmony) are spread across thousands of note events. This project builds the smallest complete version of the problem: a recurrent network that receives a categorical mood identifier together with a window of musical context and predicts the next event.

The task is conditional generation of solo piano music in a symbolic format. Given one of five mood labels and a tempo value, the system should return a short MIDI excerpt. Two practical constraints shape the work. The MAESTRO collection [3] offers 1,276 virtuoso piano performances in MIDI form but no emotion annotations, so mood labels have to be constructed. The available compute is a single Google Colab GPU session, which limits both the amount of data and the number of experiments that can be run.

The objectives are to:

- convert MAESTRO MIDI files into a compact event-token sequence that preserves note timing and dynamics;
- derive transparent weak mood labels from measurable features of each performance;
- train an LSTM that predicts the next event from a context window and a mood embedding;
- decode sampled events into a playable MIDI file for a chosen mood and tempo; and
- state precisely what the resulting experiment does and does not demonstrate.

The contribution is a reproducible end-to-end prototype together with an evidence-based account of its behaviour; no new learning method is proposed. The headline finding is negative. The labelling rules placed every one of the 300 sampled performances in a single class, so the mood input carried no information during training, and the tempo argument of the generator turned out to affect only file metadata. Both problems are traced to specific parts of the implementation. The validation results (loss 1.91, next-event accuracy 46.1%) are reported as a measure of sequence modelling only, on a subset that covers 23.5% of the archive.

The remainder of the paper is organised as follows. Section II reviews recurrent music models and emotion-conditioned generation. Section III describes the data pipeline, labelling rules, tokenisation, model, and generation procedure. Section IV presents and interprets the measurements. Section V concludes and lists the changes required for a valid mood-controlled generator.

II. Literature Review

A. Recurrent Models for Symbolic Music

Recurrent networks were an early choice for music because notes form an ordered sequence, but plain recurrent units lose information over long spans, which is one reason their output tends to lack global form [4]. The long short-term memory unit addresses this with gated, additive state updates [5], and the later addition of forget gates lets the cell reset itself [6]. Eck and Schmidhuber trained LSTMs on blues improvisation and reported that the network learned the chord form and kept to it over extended output [7]. Boulanger-Lewandowski et al. paired a recurrent network with restricted Boltzmann machines to model simultaneous notes in piano-roll frames [8], and Graves showed that an LSTM trained to predict the next element can generate complex sequences

with long-range structure when it is sampled step by step [9]. Two surveys organise the field by representation, architecture, and evaluation [2], [1]; the design in this project is a standard configuration of the kind they cover: discrete tokens, an embedding layer, stacked recurrent layers, and a softmax over the vocabulary.

B. Performance Representations and Piano Data

Score-based encodings hide the timing and dynamics that distinguish one performance from another. Oore et al. therefore modelled MIDI performances directly, using an event vocabulary of note-on, note-off, time-shift, and velocity events and an LSTM that predicts the next event [10]. The tokenisation used here follows the same idea at a coarser time resolution. The MAESTRO dataset [3] supplies material for such models: piano performances from the International Piano-e-Competition with note-level MIDI, distributed with a train/validation/test split that keeps each composition within one subset. Version 3.0.0 contains 1,276 performances, about 198.7 hours, and roughly 7.04 million notes [11]. On the same data, the Music Transformer replaces recurrence with relative self-attention [12], [13] to model structure over longer spans; the LSTM in the present work is a deliberately simpler baseline.

C. Emotion-Conditioned Generation

Psychological models describe musical emotion either as categories or as positions in a valence–arousal plane [14], and perceptual research connects it to acoustic cues such as tempo, mode, and loudness [15]. Experiments on happy–sad judgements indicate that tempo and mode both contribute [16], [17]. These findings motivate rule-based labelling, but they describe tendencies in listener responses and not deterministic mappings. Several generators attach emotion to learned models. SentiMozart classifies facial expressions and conditions a stacked LSTM on the predicted category [18]. Ferreira and Whitehead steered an mLSTM through sentiment-sensitive units and confirmed the result with a user study in which listeners recognised positive and negative pieces, although the negative pieces were more ambiguous [19]. EMOPIA provides 1,087 pop-piano clips from 387 songs with clip-level emotion labels from four annotators [20], and Sulun et al. condition a transformer on continuous valence and arousal values, reporting better results than control-token conditioning [21]. In these systems the emotion signal comes from a classifier, human annotation, or an external data source. The prototype described here instead substitutes hand-written rules over the same MIDI features, which keeps the pipeline self-contained but removes any assurance that a label corresponds to a perceived mood.

D. Key Estimation, Sampling, and Evaluation

The mode estimate used for labelling correlates a duration-weighted pitch-class histogram with major and minor key profiles. The constants in the notebook appear to match the probe-tone profiles of Krumhansl and Kessler [22], although the notebook cites no source and describes the procedure as a rough heuristic. Sampling rules matter once a model continues its own output: temperature scaling and truncation trade diversity against coherence [23], and a model trained only on ground-truth contexts can drift when it conditions on its own samples [24]. Finally, evaluation remains an open problem. Likelihood on held-out sequences measures how well a model predicts data and says little about musical quality or controllability, which calls for objective feature statistics, listening tests, or both [25]. Table I summarises the works most relevant to this project.

TABLE I. LITERATURE REVIEW SUMMARY

Ref.	Work and main contribution	Relevance to this project
[7]	LSTM trained on blues improvisation; learns the chord form and keeps it over long output	Motivates LSTMs for music sequences
[10]	LSTM over MIDI performance events (note-on, note-off, time shift, velocity)	Source of the event-token idea; coarser grid here
[3]	MAESTRO: aligned piano audio and MIDI with a composition-aware split	Data source; official split not used
[12]	Music Transformer: relative self-attention on piano performances	Candidate replacement for the LSTM
[18]	SentiMozart: facial-expression classifier drives a stacked LSTM	Emotion label supplied by a classifier
[19]	mLSTM steered by sentiment-sensitive units; user study of generated pieces	Label control checked by listeners (not done here)
[20]	EMOPIA: 1,087 pop-piano clips with human emotion labels	Possible replacement for heuristic labels
[21]	Transformer conditioned on continuous valence and arousal	Alternative to five discrete moods
[25]	Objective metrics for evaluating generative music systems	Evaluation beyond loss and accuracy

III. METHODOLOGY

A. System Overview

Fig. 1 summarises the pipeline. All stages run in one Google Colab notebook. TensorFlow 2.21.0 with the Keras interface [26] builds and trains the network, and the pretty_midi library [27] parses and writes MIDI files. A random seed of 42 is set for Python, NumPy, and TensorFlow. The notebook confirmed that one GPU was available but did not print its model. Its saved artefacts are the weak-label table, the best and final model files, the training-loss plot, an evaluation-metrics file, and one generated MIDI file. The Streamlit interface mentioned at the end of the notebook has not been built and is outside the scope of this paper.

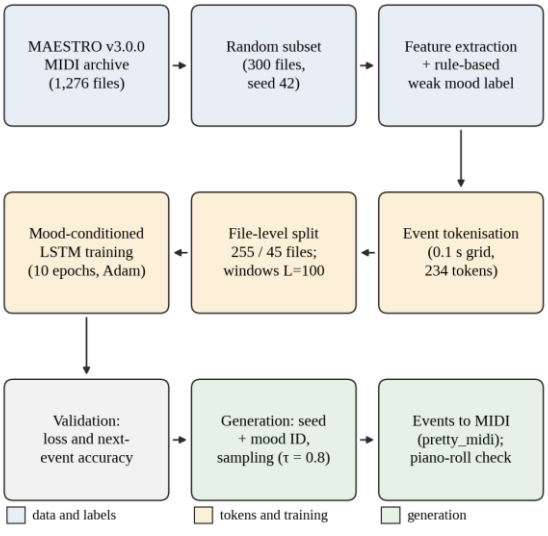


Fig. 1. Workflow of the mood-conditioned generator, from the MAESTRO MIDI archive to a generated MIDI file.

B. Dataset and Subset Selection

The MIDI-only archive of MAESTRO version 3.0.0 [3], [11] was downloaded from the dataset's public storage location. The archive (58.42 MB) contains 1,280 entries, of which 1,276 are MIDI files, matching the published performance count. To keep prototyping practical, the file list

was shuffled with seed 42 and the first 300 files (23.5% of the archive) were retained. Feature extraction succeeded for all 300 files with no failures, and each file produced more than 100 event tokens (Table II).

The 300 files were divided at random into 255 training and 45 validation performances, a 15% validation fraction. Because the split is made at file level, no window from a validation performance appears in the training data. The split does not use the composition-aware partition distributed with MAESTRO, which is designed to keep every composition in a single subset [3], [11]. As MAESTRO contains several performances of the same works, a randomly chosen validation performance may share its composition with training material, and validation scores may therefore be somewhat optimistic. The notebook does not record composition metadata, so the extent of any overlap is unknown. The vocabulary was built from the training portion only; any validation token missing from it is dropped during windowing, and the number of dropped tokens was not reported.

TABLE II. DATA AND PREPROCESSING SUMMARY

Item	Value
Archive file	maestro-v3.0.0-midi.zip, 58.42 MB
Archive entries / MIDI files	1,280 / 1,276
Files sampled (random, seed 42)	300
Usable / failed after feature extraction	300 / 0
Files with more than 100 event tokens	300
Training / validation files	255 / 45
Window length / stride	100 / 10 events
Training windows	487,306
Validation windows	121,670
Vocabulary size (training split)	234
Mood classes present in the data	1 of 5 (energetic)

C. Weak Mood Labelling

MAESTRO carries no emotion labels, so five categories (happy, sad, calm, energetic, suspenseful) are assigned by fixed rules over four features computed for each performance. Tempo is the median of the tempo values stored in the file, with 110 BPM used when none is present. Note density is the number of non-drum notes divided by the performance duration (at least one second). Pitch range is the difference between the 90th and 10th percentiles of note pitch. The mode is estimated by correlating a pitch-class histogram with key profiles. Let **c** be the 12-element histogram of pitch classes weighted by note duration (with a floor of 0.05 s per note), and let P_{maj} and P_{min} be the major and minor key profiles. For each mode the best Pearson correlation over the twelve rotations of its profile is

$$r_{\text{mode}} = \max_{k=0,\ldots,11} \operatorname{corr}(\operatorname{roll}(P_{\text{mode}}, k), \mathbf{c}) \quad (1)$$

and the performance is estimated to be major when $r_{\text{maj}} \geq r_{\text{min}}$. The notebook itself calls this a very rough heuristic and not a reliable key detector.

The rules are applied in the order of Table III and the first match decides the label. Because the density test of the first rule comes before everything else, any performance with at least 3.5 notes per second is labelled energetic regardless of tempo or mode. The thresholds are fixed values in the implementation; no source or calibration against MAESTRO is given, and the notebook does not print the distribution of the extracted features. The tempo and mode logic is consistent with the perceptual findings reviewed earlier [16], [17], whereas the density and pitch-range criteria are design choices of the prototype.

TABLE III. WEAK MOOD-LABEL RULES (FIRST MATCH WINS)

Order	Condition	Label
1	tempo ≥ 125 BPM, or density ≥ 3.5 notes/s	energetic
2	tempo ≤ 78 BPM and density ≤ 2.5 notes/s	calm
3	estimated major mode and tempo ≥ 95 BPM	happy
4	estimated minor mode and tempo ≤ 115 BPM	sad
5	otherwise, pitch range ≥ 24 semitones or density ≥ 2.8 notes/s	suspenseful
6	otherwise, tempo < 100 BPM / tempo ≥ 100 BPM	calm / happy

D. Event Tokenisation

Each non-drum note contributes three events. Start and end times are rounded to a 0.1 s grid, and every note lasts at least one grid step. A note yields a velocity token, a note-on token carrying the MIDI pitch, and a note-off token at its end time. Events are sorted by time and, within one instant, by priority: note-offs first, then velocity tokens, then note-ons. Silence between consecutive events is encoded with time-shift tokens of 1 to 50 grid steps (0.1 to 5.0 s), and longer gaps are split across several tokens. MIDI velocity v is quantised into eight bins of width 16 by

$$b(v) = \min(\max((v - 1) \operatorname{div} 16, 0), 7) \quad (2)$$

where div denotes integer division. Table IV lists the token families. The breakdown is inferred from the printed vocabulary size of 234, which equals 50 + 8 + 88 + 88, that is, every time shift, every velocity bin, and the 88 keys of a piano; the notebook prints only the total.

TABLE IV. EVENT VOCABULARY

Token family	Count	Meaning
`TIME_SHIFT_k`, k = 1…50	50	advance the clock by 0.1k s
`VELOCITY_b`, b = 0…7	8	velocity bin of the next note-on
`NOTE_ON_p`	88	key press at MIDI pitch p
`NOTE_OFF_p`	88	key release at MIDI pitch p
Total	**234**	**size printed by the notebook; split inferred**

Two properties of this scheme matter for the results. First, drums, the sustain pedal, and other control-change messages are not tokenised, so pedalling, which is central to piano expression, is absent from the representation. Second, the sort order places the velocity tokens of all simultaneous notes before their note-on tokens, as the first printed example sequence shows with its run of consecutive velocity tokens. Since decoding treats velocity as a running state, notes that start together all receive the last velocity of the group, which weakens the link between a note and its dynamics.

E. Windowing

Training examples are sliding windows over each tokenised performance. A context of 100 consecutive tokens predicts the single token that follows, and the window advances by 10 tokens. The mood identifier of the source performance is attached to every one of its windows. This produces 487,306 training and 121,670 validation windows,

about 1,911 and 2,704 windows per performance on average. With a stride that is much smaller than the window, successive windows share 90% of their tokens and are strongly correlated; the file-level split keeps this correlation from crossing into the validation set.

F. Model Architecture

The network (Fig. 2) has two inputs: the 100-token context and a mood identifier. The tokens pass through a 128-dimensional embedding and two stacked LSTM layers of 256 units, with dropout [28] of 0.3 between them. The first layer returns its full sequence so that the second can read it, and the second returns only its final hidden state $\mathbf{h}_t$. In parallel, the mood identifier m is mapped to a 16-dimensional embedding $\mathbf{e}_m$. The two vectors are concatenated,

$$\mathbf{z} = [\, \mathbf{h}_t \,;\, \mathbf{e}_m \,] \quad (272 \text{ elements}) \tag{3}$$

passed through a 128-unit ReLU layer with dropout 0.25, and projected onto the 234-token vocabulary with a softmax,

$$\mathbf{p} = \mathrm{softmax}(\, \mathbf{W}_2\, \mathrm{ReLU}(\, \mathbf{W}_1 \mathbf{z} + \mathbf{b}_1 \,) + \mathbf{b}_2 \,) \tag{4}$$

where $\mathbf{p}$ is the predicted distribution over the next event. The parameter counts in Fig. 2 agree with the standard LSTM formula of $4n(d + n + 1)$ weights for n units and d input features: $4 \cdot 256 \cdot (128 + 256 + 1) = 394{,}240$ for the first layer and $4 \cdot 256 \cdot (256 + 256 + 1) = 525{,}312$ for the second.

Fusing the label late, after the sequence has been encoded, keeps the model simple and adds only 80 parameters, but it also limits what the label can do. The mood vector can shift the final classifier's output probabilities; it cannot change how the LSTM layers read the context. A stronger conditioning design would inject the mood earlier, for example as a prefix token or as an extra input at every time step.

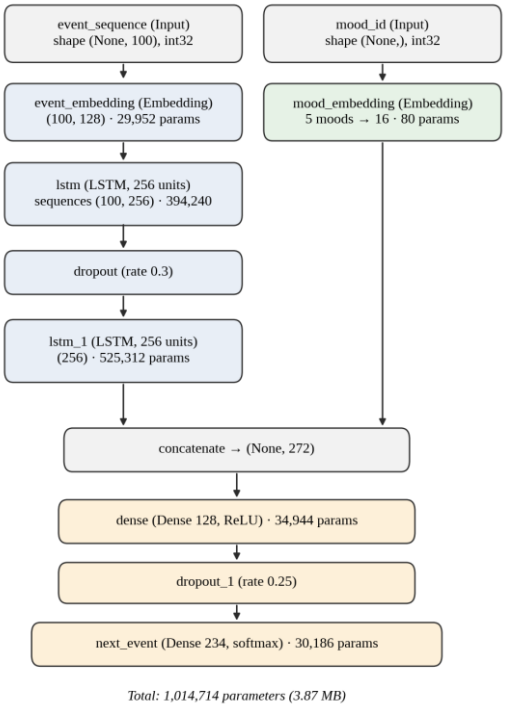


Fig. 2. System architecture of the mood-conditioned LSTM with output shapes and parameter counts as reported by the notebook.

G. Training Procedure

The model is trained to minimise the sparse categorical cross-entropy over N windows,

$$L = -(1/N) \sum_{n=1}^{N} \log p_\theta(\, y_n \mid c_n, m_n \,) \tag{5}$$

where c_n is the context, m_n the mood, y_n the target token, and p_θ the softmax output. Optimisation uses Adam [29] with an initial learning rate of 10^{-3} and batches of 64 windows, for at most ten epochs. Three callbacks watch the validation loss: early stopping with a patience of three epochs, checkpointing of the best model, and a halving of the learning rate after two epochs without improvement. Table V gives the full configuration.

TABLE V. TRAINING CONFIGURATION

Setting	Value
Optimiser	Adam, initial learning rate 10^{-3}
Loss / metric	Sparse categorical cross-entropy / sparse categorical accuracy
Batch size, epochs	64 windows; at most 10 epochs (7,615 steps per epoch)
Dropout	0.3 between the LSTM layers; 0.25 before the output layer
Callbacks	Early stopping (validation loss, patience 3, restore best weights); checkpoint on best validation loss; learning rate × 0.5 after 2 stagnant epochs (minimum 10^{-5})
Epochs completed	10
Hardware / software	Google Colab, one GPU (model not printed); TensorFlow 2.21.0
Random seed	42 (Python, NumPy, TensorFlow)
Training time	190–261 s per epoch; about 34 min in total

H. Generation and MIDI Decoding

Generation is autoregressive. The seed is the last 100 tokens of the longest validation performance, whose weak label is energetic, so every excerpt continues from a real musical context. At each step the model returns a distribution $\mathbf{p}$ over the vocabulary, which is re-weighted by a temperature τ before a token is drawn:

$$q_i = \exp(\, \log p_i / \tau \,) \,/\, \textstyle\sum_j \exp(\, \log p_j / \tau \,) \tag{6}$$

Probabilities are clipped to at least 10^{-12} for numerical safety. With $\tau = 0.8$ the distribution is sharper than the model's raw output, trading some variety for coherence [23]. The drawn token is appended to the context, the oldest token is dropped, and the loop repeats for 500 steps. The selected mood enters only through its identifier.

The sampled tokens are turned into notes by a small state machine. Time-shift tokens advance a clock, velocity tokens set the current velocity, note-on tokens open a note, and note-off tokens close the oldest open note of the same pitch. The decoded velocity of a bin b is

$$\hat{v} = \mathrm{clip}(\, (\, b + 0.5 \,) \cdot 127 / 8,\ 1,\ 127 \,) \tag{7}$$

truncated to an integer. Notes that are never closed are ended at the final time of the excerpt, and zero-length notes are discarded. The generator also accepts a tempo in beats per minute, which is passed to pretty_midi as the initial tempo of the output file. Note times, however, are already expressed in seconds on the 0.1 s grid. A check made while preparing this report (writing identical notes at 60, 90, and 140 BPM and reading the files back) returned the same note times to within 1 ms. The tempo argument therefore changes the tempo marking stored in the file but not when the notes sound, so in the implementation as written the tempo setting does not control the pace of the generated music.

I. Evaluation Protocol

Two quantities are computed on the 121,670 validation windows with the final weights: the cross-entropy loss and the top-1 next-event accuracy. Perplexity,

$$\mathrm{PPL} = \exp(L) \qquad (8)$$

is derived from the loss for interpretation. Generated output is assessed only by its note count, duration, and a piano-roll plot. No separate test set, per-mood analysis, comparison of musical features across moods, or listening test was carried out, so the protocol measures sequence modelling and not mood control.

IV. Results and Discussions

A. Outcome of the Weak-Labelling Step

The rule-based labelling did not produce the intended five-class distribution. As Fig. 3 shows, all 300 sampled performances were labelled energetic and the other four classes received no examples. The same holds for the training and validation windows: every window carries mood identifier 3.

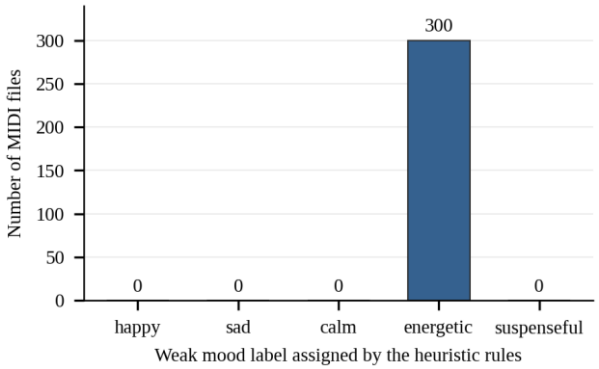


Fig. 3. Weak mood labels assigned to the 300 sampled MAESTRO performances (notebook output: Counter({'energetic': 300})).

The notebook does not print the extracted features, so the cause cannot be confirmed from its output, but the first rule offers a plausible explanation. It assigns the energetic label to any performance with at least 3.5 notes per second. The statistics published for MAESTRO version 3.0.0, 7.04 million notes in 198.7 hours [11], imply a dataset-wide mean of roughly 9.8 notes per second, almost three times the threshold. A mean is not a per-piece distribution, so this remains a hypothesis; printing the density histogram of the 300 files would settle it.

The consequences are structural. With a single class in the data, the mood input is constant and the loss cannot reward the network for using it. The embedding rows of the four unseen moods never receive a gradient and, under the Adam optimiser, are expected to keep their random initial values; this follows from the code and the label counts and was not checked by inspecting the weights. Requesting "calm" at generation time therefore feeds the classifier an untrained random vector, and the output cannot be read as calm music. The experiment as run provides no evidence for or against mood control.

B. Training Behaviour

Fig. 4 and Table VI show the training run. Training loss fell from 3.0780 to 1.9366 and validation loss from 2.7302 to 1.9105 over ten epochs, while training accuracy rose from 25.19% to 44.74%. Both curves decreased in every epoch, and the gains shrank steadily: validation loss dropped by 0.3340 between epochs 1 and 2 but by only 0.0171 between epochs 9 and 10. The model had therefore not fully converged when the epoch limit was reached. Because validation loss improved at every epoch, neither early stopping nor the learning-rate reduction would have been triggered, although the learning-rate value itself was not printed.

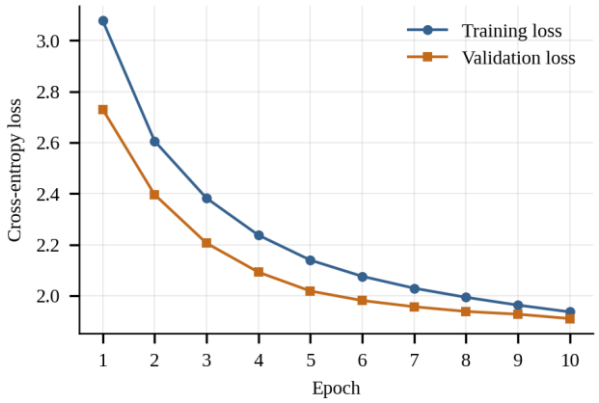


Fig. 4. Training and validation cross-entropy loss over ten epochs, re-plotted from the epoch logs of the training cell.

Validation loss lies below training loss throughout, which is not by itself a sign of data leakage. Dropout is active only during training, and the training figures are running averages over an epoch in which the weights were still improving, whereas validation is computed afterwards. The gap narrows from 0.348 in the first epoch to 0.026 in the last. Even so, a composition-level overlap between the randomly chosen training and validation files, discussed in Section III, cannot be ruled out from the available outputs. No overfitting is visible within ten epochs; a longer run would be needed to find where validation loss stops improving.

TABLE VI. Performance Comparison

Stage	Train loss	Train acc.	Val. loss	Val. acc.
Uniform guess over 234 tokens	–	–	5.455	0.43%
Epoch 1	3.0780	25.19%	2.7302	n/a
Epoch 3	2.3823	35.92%	2.2061	n/a
Epoch 5	2.1396	41.09%	2.0185	n/a
Epoch 10	1.9366	44.74%	1.9105	n/a
Final evaluation	–	–	1.9105	46.09%

Epoch rows are taken from the training log. Validation accuracy per epoch is cut off in the supplied print-out (n/a). The uniform row is a computed reference (loss = ln 234), not a trained model.

C. Final Validation Metrics

Evaluation on the 121,670 validation windows gave a loss of 1.9105 and a next-event accuracy of 46.09%. By (8) this corresponds to a perplexity of 6.76. A model that guessed uniformly over the 234 tokens would reach a loss of ln 234 = 5.455 and an accuracy of 0.43% (Table VI), so the network has clearly learned local regularities of the token stream. Three limits apply to this result. No stronger baseline, such as a unigram or bigram model, was computed, so the margin over simple statistics is unknown. Token types differ greatly in difficulty: tokens that are structurally constrained, such as the note-on that normally follows a run of velocity tokens, are plausibly easier to predict than the choice of pitch, so top-1 accuracy over all tokens blends easy and hard decisions. Finally, a single training run and a single 45-file validation sample were used, with no repeated seeds or confidence intervals. The figures should not be compared with published results, which use different data, vocabularies, and time resolutions.

D. Generated Sample

Fig. 5 is the notebook's piano-roll plot of the one excerpt that was generated (mood "calm", tempo 90 BPM, $\tau = 0.8$). The 500 sampled events decoded into 153 notes lasting 7.3 s, and generation took 44 s at about 12 steps per second. Reading the plot, the excerpt spans roughly MIDI pitches 22 to 102 and contains several long notes that run to the end. Under the decoding rule above, a note with no matching note-off is closed at the final time, so these long notes suggest that the sampler often opened notes it did not close within 500 events; counting unmatched note-ons in the token list would confirm this.

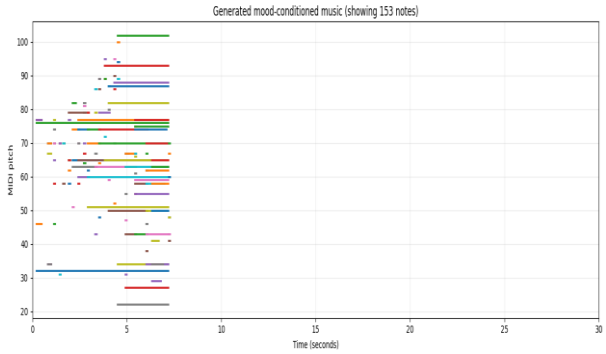


Fig. 5. Piano roll of the generated excerpt (mood "calm", 90 BPM requested), cropped from the notebook output; each bar is one note.

The excerpt also fails the notebook's own definition of calm. Its average density is 153 / 7.3 ≈ 21 notes per second, roughly twice the dataset-wide mean quoted above and far above the 2.5 notes per second allowed for the calm class in Table III; applying the rules of Table III to the excerpt would label it energetic. This follows arithmetically from the reported note count and duration. It does not show that the model ignores the mood input, because that input was untrained for "calm", and it does not show how a correctly trained model would behave. It does show that the delivered sample cannot serve as an example of calm music. Only one mood was generated; the cells for the other moods remained commented out, so no comparison between moods exists.

E. What the Results Establish

Table VII separates what was observed from what was derived or inferred and from what remains unknown.

TABLE VII. OBSERVATIONS

Observation	Basis	Status
All 300 sampled files received the energetic label	Notebook output of the labelling cell	Observed
Moods other than energetic never occur in the training data	Follows from the label counts and window construction	Derived
Validation loss 1.9105 and accuracy 46.09% after ten epochs	Training log and evaluation cell	Observed
Validation loss was still decreasing at epoch 10	Epoch logs	Observed
Embeddings of unused moods remain at their initial values	Gradient flow under Adam; weights not inspected	Inferred
Generated "calm" excerpt has about 21 notes per second	153 notes in 7.3 s	Derived
The tempo argument does not change note timing	Independent pretty_midi check at 60, 90, 140 BPM	Verified outside the notebook
The model controls mood	No data for it	Not established
Generated music is perceived as the requested mood	No listening test	Not established

In short, the work establishes a functioning path from MIDI files to a trained next-event predictor and back to MIDI. It does not establish mood control, perceived emotion, tempo control, or musical quality.

F. Limitations

- **Labels.** The mood labels are uncalibrated heuristics and collapsed to a single class; they were never validated against listeners.
- **Data and protocol.** Only 23.5% of the archive was used, in one run, with a random file-level split, no test set, and a validation set that also drove the callbacks.
- **Representation.** The 0.1 s grid, the 5 s maximum shift, the omission of pedal events, and the loose coupling of velocity tokens to notes remove expressive detail.
- **Context length.** The 100-event window is short; in the generated sample 500 events spanned 7.3 s, so 100 events cover on the order of one to two seconds, which limits long-range structure.
- **Generation.** Training on ground-truth contexts and sampling from the model's own output can cause drift [24], and unmatched note-ons produced sustained notes in the sample.
- **Control.** The tempo argument changes metadata only, and mood conditioning is untested.
- **Evaluation and output.** Loss and accuracy do not measure musical quality [25], and the output is symbolic MIDI, not audio.

V. Conclusion and Future Work

This project assembled a complete pipeline from MAESTRO MIDI files to a trained LSTM and a decoded MIDI excerpt. The network, with 1,014,714 parameters, learned to predict the next event with a validation loss of 1.9105 and an accuracy of 46.09% after ten epochs on 300 performances, against reference values of 5.455 and 0.43% for uniform guessing. The central aim, a generator that responds to a requested mood, was not achieved or demonstrated. The heuristic labelling collapsed to one class, which left four of the five mood embeddings untrained, and the tempo argument was found to alter only file metadata. The prototype is therefore best regarded as a working data and training scaffold, and the numbers it produced describe sequence modelling only.

The following changes would turn it into a testable mood-controlled generator:

- Recalibrate the labelling rules against the measured feature distributions, for example with percentile thresholds, report class counts before training, and stop when any class is empty or too small.

- Replace or validate the heuristic labels with human annotation such as EMOPIA [20], or consider continuous valence–arousal conditioning [21], [14].
- Use the composition-aware MAESTRO split with a held-out test set, train on the full archive, and repeat runs over several seeds.
- Implement tempo control by scaling generated onsets or by adding a tempo token, and inject the mood earlier in the network, for example as a prefix token.
- Enrich the tokenisation with pedal events and a finer time grid, and bind each velocity token to its note.
- Evaluate several samples per mood by comparing density, pitch range, and mode statistics across moods, and run a small listening study [25], [19].
- Compare against a Transformer with relative attention [12], and then build the planned Streamlit interface.

REFERENCES

[1] S. Ji, J. Luo, and X. Yang, "A comprehensive survey on deep music generation: Multi-level representations, algorithms, evaluations, and future directions," 2020, arXiv:2011.06801. [Online]. Available: https://arxiv.org/abs/2011.06801

[2] J.-P. Briot, G. Hadjeres, and F.-D. Pachet, *Deep Learning Techniques for Music Generation*. Cham, Switzerland: Springer, 2020.

[3] C. Hawthorne *et al.*, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[4] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[6] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Comput.*, vol. 12, no. 10, pp. 2451–2471, Oct. 2000.

[7] D. Eck and J. Schmidhuber, "Finding temporal structure in music: Blues improvisation with LSTM recurrent networks," in *Proc. 12th IEEE Workshop Neural Netw. Signal Process.*, 2002, pp. 747–756.

[8] N. Boulanger-Lewandowski, Y. Bengio, and P. Vincent, "Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation and transcription," in *Proc. 29th Int. Conf. Mach. Learn. (ICML)*, 2012.

[9] A. Graves, "Generating sequences with recurrent neural networks," 2013, arXiv:1308.0850. [Online]. Available: https://arxiv.org/abs/1308.0850

[10] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Comput. Appl.*, vol. 32, no. 4, pp. 955–967, 2020.

[11] Magenta, Google, "The MAESTRO dataset, version 3.0.0." [Online]. Available: https://magenta.tensorflow.org/datasets/maestro

[12] C.-Z. A. Huang *et al.*, "Music Transformer: Generating music with long-term structure," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[13] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[14] J. A. Russell, "A circumplex model of affect," *J. Pers. Soc. Psychol.*, vol. 39, no. 6, pp. 1161–1178, 1980.

[15] P. N. Juslin and P. Laukka, "Expression, perception, and induction of musical emotions: A review and a questionnaire study of everyday listening," *J. New Music Res.*, vol. 33, no. 3, pp. 217–238, 2004.

[16] L. Gagnon and I. Peretz, "Mode and tempo relative contributions to 'happy–sad' judgements in equitone melodies," *Cogn. Emot.*, vol. 17, no. 1, pp. 25–40, 2003.

[17] S. Dalla Bella, I. Peretz, L. Rousseau, and N. Gosselin, "A developmental study of the affective value of tempo and mode in music," *Cognition*, vol. 80, no. 3, pp. B1–B10, 2001.

[18] R. Madhok, S. Goel, and S. Garg, "SentiMozart: Music generation based on emotions," in *Proc. 10th Int. Conf. Agents Artif. Intell. (ICAART)*, vol. 2, 2018, pp. 501–506.

[19] L. N. Ferreira and J. Whitehead, "Learning to generate music with sentiment," in *Proc. 20th Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, Delft, The Netherlands, 2019, pp. 384–390.

[20] H.-T. Hung, J. Ching, S. Doh, N. Kim, J. Nam, and Y.-H. Yang, "EMOPIA: A multi-modal pop piano dataset for emotion recognition and emotion-based music generation," in *Proc. 22nd Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, 2021, pp. 318–325.

[21] S. Sulun, M. E. P. Davies, and P. Viana, "Symbolic music generation conditioned on continuous-valued emotions," *IEEE Access*, vol. 10, pp. 44617–44626, 2022, doi: 10.1109/ACCESS.2022.3169744.

[22] C. L. Krumhansl and E. J. Kessler, "Tracing the dynamic changes in perceived tonal organization in a spatial representation of musical keys," *Psychol. Rev.*, vol. 89, no. 4, pp. 334–368, 1982.

[23] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[24] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015.

[25] L.-C. Yang and A. Lerch, "On the evaluation of generative models in music," *Neural Comput. Appl.*, vol. 32, no. 9, pp. 4773–4784, 2020, doi: 10.1007/s00521-018-3849-7.

[26] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2016, pp. 265–283.

[27] C. Raffel and D. P. W. Ellis, "Intuitive analysis, creation and manipulation of MIDI data with pretty_midi," in *Proc. 15th Int. Soc. Music Inf. Retrieval Conf. Late Breaking and Demo Papers*, 2014.

[28] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.

[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.